Account Ledger Core: Architecture & Trade-offs

Feroz Khatri · 1 October 2026

Shape of the implementation

Every entry carries two dates: a **booking day** (when the system learned about it) and a **value date** (when it counts). Balances are always computed by value date. Money is held as `bigint` minor units, so there is no floating-point arithmetic, and the only rounding is exact half-even on interest. The ledger is an append-only log with per-account, per-value-day totals, so a balance costs O(days), not O(entries). Authorizations are held in a separate projection (`AuthorizationBook`) and never touch the ledger. A single-threaded engine validates each event with guard clauses, dispatches it to a small handler per event type, and records every outcome, rejections included. At end of day it re-checks overdraft fees across all value days and, on the last day, capitalizes interest calculated from the final value-dated balances. Fee idempotency is derived from the FEE entries already in the ledger, so there is no second copy of that state that could drift.

1. Append-only at scale

What breaks first at 100× volume: the end-of-day batch. For every account, every night, `closeDay`:

1. rebuilds the closing balance for every day in the window;
2. scans **all** of the account's entries to find days already charged a fee;
3. scans **every authorization in the bank** to total the account's active holds, because `activeHoldsMinor` filters the whole book.

Steps 1 and 2 grow linearly. Step 3 costs accounts × all authorizations, which is quadratic. 100× accounts makes it about 10,000× slower, long before the ledger's size itself becomes a problem. The same scan also runs on every authorization request, so decision latency at the point of sale grows with the size of the whole bank. That is the first failure a customer would notice.

Where state grows without limit:

- the entry log, the per-account entry arrays and the source-event index (append-only, never pruned);
- the per-value-day totals, which keep a key for every day ever used although only the window is ever queried;
- the engine's outcome log and duplicate-event set (every event ID ever seen, forever);
- the authorization book (declined and settled authorizations are kept and scanned forever);
- the fee re-check window. How far back it looks is limited only by how far back a value date may reach, which is currently the whole window.

The cheapest structural change: a closed-period checkpoint. Once a day falls behind a back-dating horizon (T−N business days), fold it into a per-account snapshot: the carried-forward balance plus the days already charged a fee. Value dates earlier than the checkpoint are no longer accepted. Balances then start from the snapshot, the fee re-check only covers days after the horizon, and fee idempotency reads a small set instead of scanning entries. Nothing is deleted: entries move to cold storage and remain the audit record. The engine and event model don't change. The design gains one new rule, "no value date behind the checkpoint", which is also the control proposed in §2. Two smaller changes belong with it: index active holds by account, removing an authorization when it leaves APPROVED; and limit duplicate detection to IDs within the horizon, with older IDs checked in cold storage.

2. Value-dated entries in production

Value dating separates *when the bank knew* from *when it counts*. In a UAE-licensed bank that creates obligations in several areas:

- **Customer money and conduct.** A back-dated entry changes history: interest, overdraft fees and minimum-balance charges for past days are now different. This implementation shows the problem: fees charged when they were correct stay on the account after a later reversal restores the balance. CBUAE's consumer-protection expectations on fair, transparent charges require an explicit refund policy for these cases. Leaving it to processing order is not acceptable.
- **Statements and disputes.** A statement already issued for the affected days becomes wrong. The bank needs a documented policy (a corrected statement, or an adjustment shown on the next one), and customer-facing records need both dates so that disputes can be resolved.
- **Regulatory and financial reporting.** Returns to CBUAE, liquidity and reserve positions, and general-ledger postings are taken at cut-off times. A back-dated entry that crosses a cut-off changes figures that have already been reported. Every reported figure must be reproducible as it was known at the time (a booking-date view) as well as in its restated form. That is why each entry here carries both dates rather than overwriting one.

- **Accounting periods.** Back-dating into a closed month or year changes audited figures. For profit-sharing (Islamic) accounts, the profit for a closed period has already been distributed and cannot simply be recalculated.
- **Financial crime.** Back-dating can disguise when a transaction happened: moving it before a limit reset, out of a monitoring window, or out of an overdraft. Transaction monitoring must look at booking time, not only value date.

The control to add before go-live: a back-dating limit, with maker-checker approval above it. Automated channels may only post value dates within N business days of the booking day and never into a closed accounting period. Anything older goes through an operations workflow that needs a second approver and a reason code. Each of these postings appears on a daily exception report to Finance and Compliance, and the posting is logged against both dates. This is the same horizon as the checkpoint in §1, so one control addresses both the regulatory risk and the scaling risk.

3. Authorization lifecycle

In this model an authorization is created as `APPROVED` or `DECLINED`, and only an approved authorization can become `SETTLED`. A settlement for less than the hold also ends it as SETTLED, and the whole hold is released (a single-capture model). Apart from a matching settlement, an authorization in this model can end in four ways:

How it ends in this model	Real-world scenario	Behaviour I would require
Declined at request. Terminal state; no hold is placed. A later settlement is rejected as `AUTHORIZATION_NOT_APPROVED`.	Not enough available funds at the point of sale, or a blocked card or account.	Return a decline reason to the scheme. Keep the record for fraud and dispute evidence. A later settlement is a force post (see the last row).
Settled for less than the hold. SETTLED; the unused remainder is released.	A final amount below the estimate: fuel, a cancelled item, a hotel deposit larger than the bill.	Release the remainder at settlement, as the model does. For split shipments, support multiple captures with a "remaining hold" balance instead of releasing everything at the first capture.
Never settled. Stays APPROVED, and the hold reduces available balance indefinitely. There is no expiry in the model.	The merchant never sends the settlement: an abandoned order, a car-rental or hotel deposit, a failed clearing batch, a merchant-side cancellation that never reached the issuer.	Holds must expire after a period set by merchant category (days for retail, weeks for travel). Expiry releases the hold and writes an audit event. Merchant cancellations should arrive as an explicit authorization reversal that is idempotent by authorization ID. That is not a ledger reversal, because no money moved.
Settlement rejected, authorization left open. Over-capture (`SETTLEMENT_EXCEEDS_HOLD`), a currency or account mismatch, or a second settlement of an already-settled authorization. The hold remains, or the attempt is refused.	Tips and currency-conversion differences (over-capture), a routing error, a resent clearing record.	Accept over-capture within a per-merchant-category tolerance and send anything above it to review. The scheme will claim the money either way, so an outright refusal only creates a reconciliation break. Detect duplicates by the clearing record's reference, not by authorization ID alone.

The model also refuses a settlement that has **no authorization**, and no money moves. Card-scheme rules often oblige the issuer to honour these (offline or below-floor-limit transactions, force posts). In production the debit should be posted and routed to disputes and chargebacks, not refused. Refusing it means the scheme is owed money that was never taken from the customer.

4. What I cut, and why

Simplification	Production risk it defers
In memory, single process, no persistence	A crash loses everything. Production needs a durable, ordered event log, transactional appends and snapshots. The replay design assumes the log already exists.
Strictly one event at a time	Two authorizations arriving at once on one account could both pass the available-balance check. Needs per-account serialisation (partitioned single writer or row locks) plus idempotency keys.
Integer days; no calendar, cut-off time, holidays or time zones	Value dates depend on UAE business days, per-currency settlement calendars and a defined end-of-day cut-off time.
No foreign exchange; fees configured per currency, only AED set	An overdrawn account in another currency raises `FEE_NOT_CONFIGURED` and is not charged. FX rates and multi-currency fee schedules are not designed.
No fee refund or re-evaluation after reversals	Customers keep paying fees whose trigger no longer exists (captured by the annotated failing test). Conduct risk.
No back-dating limit	Any caller can post into any past day of the window. See the control in §2.

Simplification	Production risk it defers
Single-capture authorizations; no expiry, authorization reversal, incremental authorization or over-capture tolerance; force posts refused	Holds that never clear, available balances wrongly reduced, scheme obligations breached (§3).
Duplicate detection by event ID only, kept forever	Memory grows without limit, and a clearing record resent with a new ID is booked twice. Needs deduplication by business reference, with a time limit.
Simple daily interest on the ledger balance	No day-count convention (Actual/365 or Actual/360), tiers, minimum balances, or profit-sharing rules.
A rejected event can never be resubmitted under the same ID	Retrying a corrected event is indistinguishable from a duplicate. Needs a documented contract separating retries from corrections.
No authentication, authorization, rate limiting or untrusted input path	The engine trusts its caller. An API in front of it needs identity, schema validation (amount parsing is already strict) and rate limits.
Plain-text report and JSON logs to stderr; no metrics or alerts	No latency percentiles, no decline, fee or late-event rates, no alerting on configuration errors. Logs carry event IDs for correlation but are not shipped or retained anywhere.